Lekcja 09: Programowanie obiektowe (OOP) w Data Science

`#lekcja` `#datascience` `#python` `#oop` `#klasy` `#obiekty`

W tej lekcji poznasz podstawy **programowania obiektowego (OOP)** w kontekście Data Science. Dowiesz się, czym są klasy i obiekty, jak tworzyć własne struktury danych, wykorzystywać dziedziczenie i kompozycję. Zrozumiesz, dlaczego OOP jest kluczowe dla tworzenia czytelnych, łatwych w utrzymaniu projektów analitycznych i jak podejście obiektowe wspiera rozwój złożonych pipeline'ów przetwarzania danych.

---

1. Wprowadzenie do OOP – klasy i obiekty

> **Definition**
>
> **Programowanie obiektowe (OOP)** to paradygmat programowania, w którym dane i funkcje operujące na tych danych są organizowane w **obiekty**. **Klasa** to szablon (przepis) definiujący strukturę i zachowanie obiektów, a **obiekt** to konkretna instancja klasy.

Wprowadzenie

W Data Science często pracujemy z wieloma funkcjami i zmiennymi, które są ze sobą powiązane. Zamiast trzymać je osobno, możemy grupować je w **klasy**. Dzięki temu kod staje się bardziej modularny, łatwiejszy do testowania i rozwijania.

Przykład z życia:

- Klasa: `Samochód` (szablon)
- Obiekty: konkretny samochód (np. `mój_samochód = Samochód("Tesla", "Model 3")`)

W Data Science możemy mieć klasę `Dataset`, która grupuje dane i metody ich przetwarzania.

Przykład 1: Podstawowa klasa – tworzenie obiektu

```
# Prosta klasa reprezentująca dataset
class Dataset:
    def __init__(self, nazwa):
        """
```

```
        Konstruktor klasy Dataset.
        __init__ to specjalna metoda wywoływana podczas tworzenia obiektu.
        'self' to referencja do aktualnego obiektu.
        """
        self.nazwa = nazwa  # Atrybut (cecha) obiektu
        self.dane = []      # Lista na dane

    def dodaj_wiersz(self, wiersz):
        """Metoda dodająca wiersz do datasetu"""
        self.dane.append(wiersz)

    def pokaz_info(self):
        """Metoda wyświetlająca informacje o datasecie"""
        print(f"Dataset: {self.nazwa}")
        print(f"Liczba wierszy: {len(self.dane)}")

# Tworzenie obiektów (instancji klasy)
ds1 = Dataset("Sprzedaż Q1")
ds1.dodaj_wiersz([100, 200, 300])
ds1.dodaj_wiersz([150, 250, 350])
ds1.pokaz_info()

# Drugi obiekt tej samej klasy
ds2 = Dataset("Sprzedaż Q2")
ds2.dodaj_wiersz([180, 220, 320])
ds2.pokaz_info()
```

Wyjaśnienie:

- `__init__` to **konstruktor** – metoda wywoływana automatycznie przy tworzeniu obiektu
- `self` reprezentuje konkretny obiekt (np. `ds1` lub `ds2`)
- `self.nazwa` i `self.dane` to **atrybuty** (zmienne) obiektu
- `dodaj_wiersz()` i `pokaz_info()` to **metody** (funkcje) obiektu

Przykład 2: Klasa z metodami obliczeniowymi

```
# Klasa reprezentująca dane liczbowe z funkcjami statystycznymi
class StatDataset:
    def __init__(self, nazwa, wartosci):
        """
        Klasa do przechowywania danych i obliczania podstawowych statystyk.
        """
        self.nazwa = nazwa
        self.wartosci = wartosci  # Lista liczb
```

```
    def srednia(self):
        """Oblicza średnią arytmetyczną"""
        return sum(self.wartosci) / len(self.wartosci)

    def minimum(self):
        """Zwraca wartość minimalną"""
        return min(self.wartosci)

    def maksimum(self):
        """Zwraca wartość maksymalną"""
        return max(self.wartosci)

    def raport(self):
        """Generuje pełny raport statystyczny"""
        print(f"=== Raport dla: {self.nazwa} ===")
        print(f"Średnia: {self.srednia():.2f}")
        print(f"Min: {self.minimum()}")
        print(f"Max: {self.maksimum()}")
        print(f"Zakres: {self.maksimum() - self.minimum()}")

# Użycie klasy
dane_sprzedazy = StatDataset("Sprzedaż miesięczna", [1200, 1500, 1350, 1800,
1600, 1700])
dane_sprzedazy.raport()

dane_temp = StatDataset("Temperatura dzienna", [18.5, 20.1, 19.8, 22.3,
21.0, 20.5, 19.2])
dane_temp.raport()
```

Wyjaśnienie:

- Każdy obiekt `StatDataset` ma własne dane i może wykonywać na nich operacje
- Metody `srednia()`, `minimum()`, `maksimum()` operują na `self.wartosci`
- Tworzymy dwa różne obiekty z różnymi danymi i wywołujemy na nich te same metody

Przykład 3: Klasa modelująca obserwacje Data Science

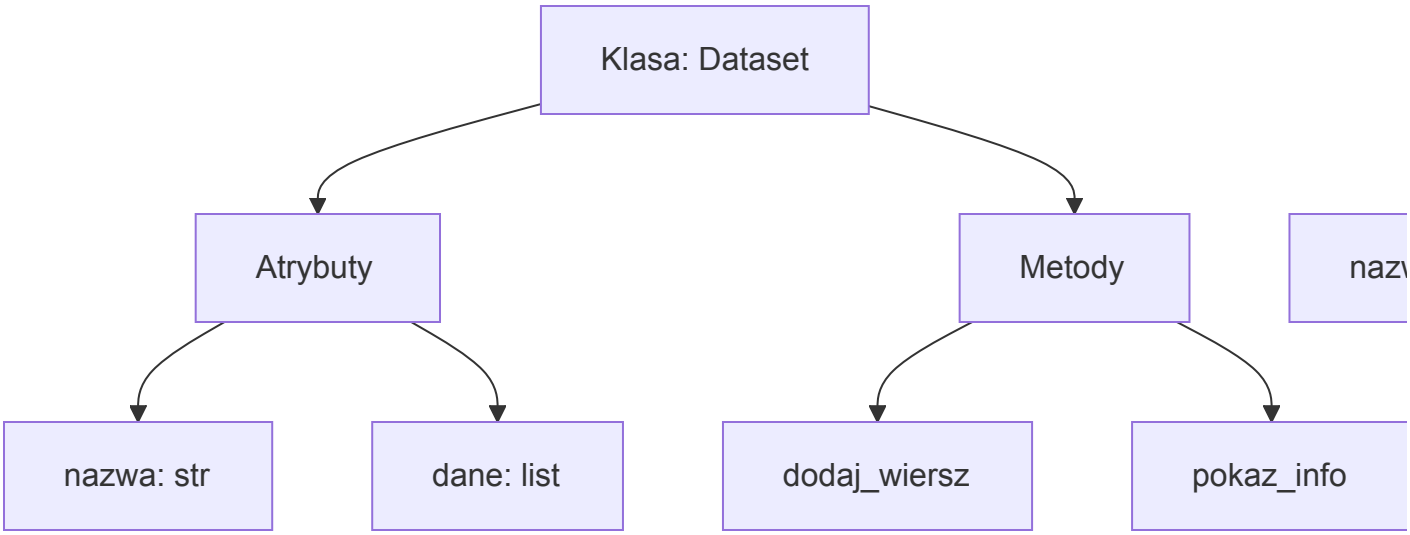
```
# Klasa reprezentująca pojedynczą obserwację w datasecie
class Obserwacja:
    def __init__(self, id, cechy, etykieta=None):
        """
        Pojedyncza obserwacja w datasecie ML.

        Parameters:
        - id: identyfikator obserwacji
```

```
        - cechy: lista lub słownik cech (features)
        - etykieta: etykieta (label) dla uczenia nadzorowanego (opcjonalna)
        """
        self.id = id
        self.cechy = cechy
        self.etykieta = etykieta

    def pokaz(self):
        """Wyświetla informacje o obserwacji"""
        print(f"ID: {self.id}")
        print(f"Cechy: {self.cechy}")
        if self.etykieta is not None:
            print(f"Etykieta: {self.etykieta}")
        else:
            print("Brak etykiety (dane unlabeled)")

    def ma_etykiete(self):
        """Sprawdza, czy obserwacja ma etykietę"""
        return self.etykieta is not None

# Tworzenie obserwacji
obs1 = Obserwacja(1, {"wiek": 25, "wzrost": 170, "waga": 65},
etykieta="zdrowy")
obs1.pokaz()

print(f"\nCzy ma etykietę: {obs1.ma_etykiete()}")

obs2 = Obserwacja(2, [1.2, 0.8, 2.3, 1.5])  # Obserwacja bez etykiety
obs2.pokaz()
print(f"Czy ma etykietę: {obs2.ma_etykiete()}")
```

Wyjaśnienie:

- Klasa `Obserwacja` reprezentuje pojedynczy wiersz danych w formacie OOP
- Obsługa zarówno słowników (`{"wiek": 25}`) jak i list (`[1.2, 0.8]`) jako cechy
- Opcjonalny parametr `etykieta` pozwala na tworzenie danych labeled/unlabeled

Wizualizacja struktury klasy



Tip

Kiedy używać klas w Data Science?

- Gdy masz wiele powiązanych funkcji i danych
- Gdy chcesz wielokrotnie używać tej samej struktury z różnymi danymi
- Gdy budujesz pipeline przetwarzania danych
- Gdy tworzysz własne transformery, estimatory, modele

Warning

Nie komplikuj kodu niepotrzebnie! Jeśli potrzebujesz tylko 1-2 funkcji, wystarczy zwykła funkcja. OOP ma sens, gdy kod staje się bardziej złożony.

---

2. Dziedziczenie – specjalizacja klas

Definition

Dziedziczenie to mechanizm, w którym nowa klasa (klasa potomna, subklasa) przejmuje atrybuty i metody z istniejącej klasy (klasy bazowej, rodzica). Pozwala to tworzyć hierarchie klas i unikać powielania kodu.

Wprowadzenie

Dziedziczenie pozwala na tworzenie specjalizowanych wersji klas. Na przykład:

- Klasa bazowa: `Model`
- Klasy potomne: `ModelRegresji`, `ModelKlasyfikacji`

Klasy potomne dziedziczą wszystkie metody z klasy bazowej, ale mogą też dodawać nowe lub modyfikować istniejące.

Przykład 1: Dziedziczenie – klasa bazowa i potomna

```
# Klasa bazowa (rodzic)
class Model:
    def __init__(self, nazwa):
        """Bazowa klasa dla wszystkich modeli ML"""
        self.nazwa = nazwa
        self.wytrenowany = False

    def trenuj(self, dane):
        """Podstawowa metoda trenowania (placeholder)"""
        print(f"Trenuję model {self.nazwa}...")
        self.wytrenowany = True

    def czy_wytrenowany(self):
        """Sprawdza, czy model jest wytrenowany"""
        return self.wytrenowany

# Klasa potomna (dziecko) dziedziczy po Model
class ModelRegresji(Model):
    def __init__(self, nazwa, typ_regresji="liniowa"):
        """
        Konstruktor klasy potomnej.
        super().__init__(nazwa) wywołuje konstruktor klasy bazowej.
        """
        super().__init__(nazwa)  # Wywołanie konstruktora rodzica
        self.typ_regresji = typ_regresji  # Nowy atrybut specyficzny dla
regresji

    def przewiduj(self, x):
        """Metoda specyficzna dla regresji"""
        if not self.wytrenowany:
            print("Błąd: Model nie jest wytrenowany!")
            return None
        # Prosta symulacja predykcji
        return 2.5 * x + 1.3

# Użycie
```

```
model_reg = ModelRegresji("Regresja Sprzedaży", typ_regresji="liniowa")
print(f"Czy wytrenowany: {model_reg.czy_wytrenowany()}")

model_reg.trenuj([1, 2, 3, 4, 5])  # Metoda z klasy bazowej
print(f"Czy wytrenowany: {model_reg.czy_wytrenowany()}")

predykcja = model_reg.przewiduj(10)  # Metoda specyficzna dla ModelRegresji
print(f"Predykcja dla x=10: {predykcja}")
```

Wyjaśnienie:

- `ModelRegresji` dziedziczy po `Model` – ma dostęp do `trenuj()` i `czy_wytrenowany()`
- `super().__init__(nazwa)` wywołuje konstruktor klasy bazowej, inicjalizując `self.nazwa` i `self.wytrenowany`
- `ModelRegresji` dodaje własny atrybut `typ_regresji` i metodę `przewiduj()`

Przykład 2: Hierarchia klas – wiele klas potomnych

```
# Klasa bazowa
class Transformer:
    def __init__(self, nazwa):
        """Bazowy transformer do przetwarzania danych"""
        self.nazwa = nazwa

    def transformuj(self, dane):
        """Metoda bazowa - do nadpisania w klasach potomnych"""
        raise NotImplementedError("Metoda transformuj() musi być
zaimplementowana!")

# Klasa potomna 1: Normalizacja
class Normalizator(Transformer):
    def __init__(self):
        super().__init__("Normalizator")
        self.min_val = None
        self.max_val = None

    def transformuj(self, dane):
        """Normalizacja min-max do zakresu [0, 1]"""
        self.min_val = min(dane)
        self.max_val = max(dane)
        zakres = self.max_val - self.min_val

        if zakres == 0:
            return [0.5] * len(dane)  # Wszystkie wartości identyczne
```

```python
        return [(x - self.min_val) / zakres for x in dane]

# Klasa potomna 2: Standaryzacja
class Standaryzator(Transformer):
    def __init__(self):
        super().__init__("Standaryzator")
        self.srednia = None
        self.odch_std = None

    def transformuj(self, dane):
        """Standaryzacja do rozkładu o średniej 0 i odchyleniu std 1"""
        self.srednia = sum(dane) / len(dane)
        wariancja = sum((x - self.srednia)**2 for x in dane) / len(dane)
        self.odch_std = wariancja ** 0.5

        if self.odch_std == 0:
            return [0.0] * len(dane)

        return [(x - self.srednia) / self.odch_std for x in dane]

# Użycie różnych transformerów
dane = [10, 20, 30, 40, 50]

norm = Normalizator()
dane_norm = norm.transformuj(dane)
print(f"Dane oryginalne: {dane}")
print(f"Po normalizacji: {[round(x, 2) for x in dane_norm]}")
print(f"Min: {norm.min_val}, Max: {norm.max_val}\n")

stand = Standaryzator()
dane_stand = stand.transformuj(dane)
print(f"Po standaryzacji: {[round(x, 2) for x in dane_stand]}")
print(f"Średnia: {stand.srednia}, Odch. std: {round(stand.odch_std, 2)}")
```

Wyjaśnienie:

- `Transformer` to klasa bazowa definiująca interfejs (metodę `transformuj()`)
- `Normalizator` i `Standaryzator` to klasy potomne implementujące różne strategie transformacji
- Każda klasa potomna ma własną implementację `transformuj()`
- Dzięki dziedziczeniu możemy tworzyć nowe transformery bez powielania kodu

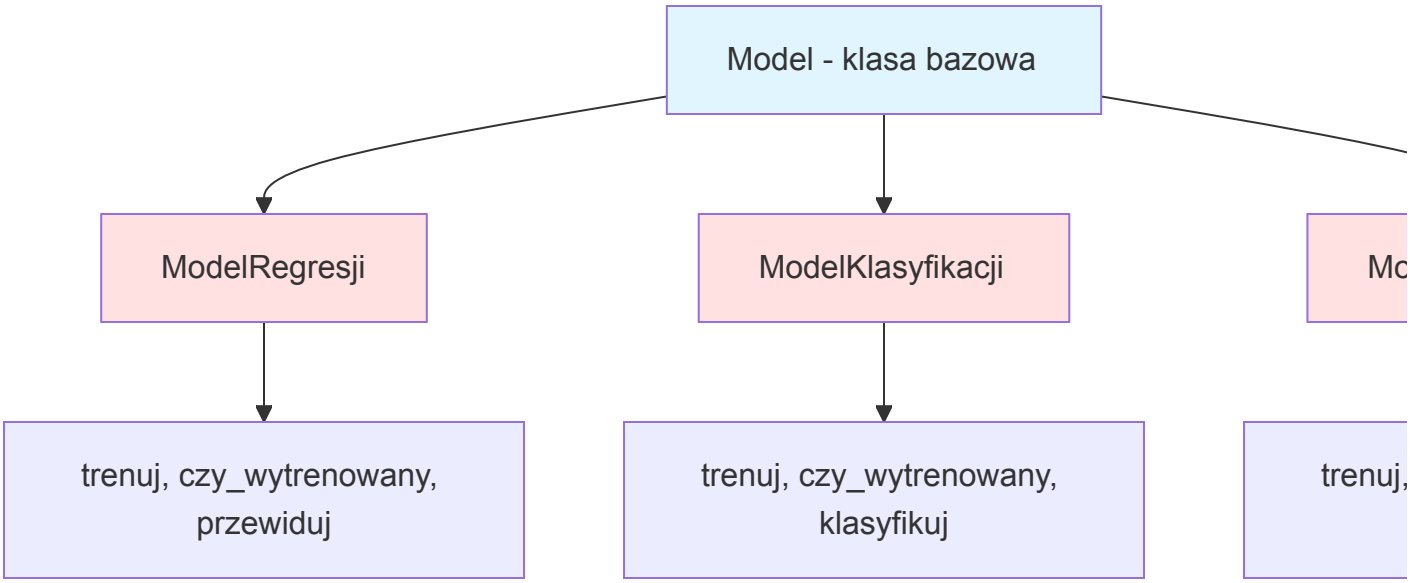
Przykład 3: Nadpisywanie metod (override)

```python
# Klasa bazowa
class Raport:
    def __init__(self, tytul):
        self.tytul = tytul

    def generuj(self):
        """Generuje podstawowy raport"""
        print(f"=== {self.tytul} ===")
        print("Brak szczegółowych danych.")

# Klasa potomna nadpisuje metodę generuj()
class RaportSzczegolowy(Raport):
    def __init__(self, tytul, dane):
        super().__init__(tytul)
        self.dane = dane

    def generuj(self):
        """Nadpisana metoda - generuje szczegółowy raport"""
        print(f"=== {self.tytul} ===")
        print(f"Liczba obserwacji: {len(self.dane)}")
        print(f"Średnia: {sum(self.dane)/len(self.dane):.2f}")
        print(f"Min: {min(self.dane)}, Max: {max(self.dane)}")

# Użycie
raport1 = Raport("Raport podstawowy")
raport1.generuj()

print()

raport2 = RaportSzczegolowy("Raport sprzedaży", [100, 150, 200, 180, 220])
raport2.generuj()  # Wywołuje nadpisaną metodę
```

Wyjaśnienie:

- `RaportSzczegolowy` **nadpisuje** metodę `generuj()` z klasy bazowej
- Metoda w klasie potomnej całkowicie zastępuje metodę z klasy bazowej
- Nadpisywanie pozwala na dostosowanie zachowania klasy do specyficznych potrzeb

Schemat dziedziczenia



Tip

Kiedy używać dziedziczenia?

- Gdy masz wiele klas o podobnej strukturze, ale różniących się szczegółami
- Gdy chcesz stworzyć hierarchię typów (np. różne typy modeli ML)
- Gdy chcesz rozszerzyć istniejącą klasę bez modyfikowania jej kodu

Warning

Nie nadużywaj dziedziczenia! Głębokie hierarchie (klasa → potomek → potomek → potomek) są trudne w utrzymaniu. Czasem lepiej użyć kompozycji.

---

3. Kompozycja – budowanie klas z innych obiektów

Definition

Kompozycja to technika, w której klasa zawiera obiekty innych klas jako swoje atrybuty. Zamiast "być" inną klasą (dziedziczenie), klasa "ma" inne obiekty jako komponenty.

Wprowadzenie

Kompozycja to alternatywa dla dziedziczenia. Zamiast tworzyć relację "jest" (is-a), tworzymy relację "ma" (has-a):

- Dziedziczenie: `Samochód` **jest** `Pojazdem`
- Kompozycja: `Samochód` **ma** `Silnik`, `Samochód` **ma** `Koła`

W Data Science kompozycja pozwala budować złożone pipeline'y z prostych komponentów.

Przykład 1: Kompozycja – pipeline przetwarzania danych

```
# Proste komponenty
class Czyszczenie:
    def __init__(self):
        self.nazwa = "Czyszczenie danych"

    def wykonaj(self, dane):
        """Usuwa wartości ujemne (symulacja czyszczenia)"""
        print(f"  [{self.nazwa}] Usuwam wartości ujemne...")
        return [x for x in dane if x >= 0]

class Normalizacja:
    def __init__(self):
        self.nazwa = "Normalizacja"

    def wykonaj(self, dane):
        """Normalizacja do zakresu [0, 1]"""
        print(f"  [{self.nazwa}] Normalizuję dane...")
        if len(dane) == 0:
            return []
        min_val = min(dane)
        max_val = max(dane)
        zakres = max_val - min_val
        if zakres == 0:
            return [0.5] * len(dane)
        return [(x - min_val) / zakres for x in dane]

class Agregacja:
    def __init__(self):
        self.nazwa = "Agregacja"

    def wykonaj(self, dane):
        """Oblicza średnią jako wynik agregacji"""
        print(f"  [{self.nazwa}] Obliczam średnią...")
        if len(dane) == 0:
            return 0
        return sum(dane) / len(dane)
```

```python
# Klasa Pipeline używa kompozycji - zawiera inne obiekty jako komponenty
class Pipeline:
    def __init__(self, nazwa):
        """
        Pipeline składa się z listy kroków (komponentów).
        Każdy komponent ma metodę wykonaj().
        """
        self.nazwa = nazwa
        self.kroki = []  # Lista komponentów

    def dodaj_krok(self, krok):
        """Dodaje komponent do pipeline'a"""
        self.kroki.append(krok)

    def uruchom(self, dane):
        """Wykonuje wszystkie kroki sekwencyjnie"""
        print(f"=== Pipeline: {self.nazwa} ===")
        wynik = dane
        for krok in self.kroki:
            wynik = krok.wykonaj(wynik)
        print(f"Wynik końcowy: {wynik}")
        return wynik

# Tworzenie pipeline'a z komponentów
pipeline = Pipeline("Przetwarzanie danych sprzedaży")
pipeline.dodaj_krok(Czyszczenie())
pipeline.dodaj_krok(Normalizacja())
pipeline.dodaj_krok(Agregacja())

# Uruchomienie pipeline'a
dane_wejsciowe = [10, -5, 20, 30, -2, 40, 50]
wynik = pipeline.uruchom(dane_wejsciowe)
```

Wyjaśnienie:

- `Pipeline` **ma** listę kroków (`self.kroki`)
- Każdy krok to osobny obiekt (`Czyszczenie` , `Normalizacja` , `Agregacja`)
- Pipeline nie dziedziczy po tych klasach – tylko je wykorzystuje
- Możemy łatwo dodawać, usuwać lub zmieniać kolejność kroków

Przykład 2: Model z wbudowanymi komponentami

```python
# Komponenty
class Preprocessor:
```

```python
    def __init__(self):
        self.nazwa = "Preprocessor"

    def przetworz(self, dane):
        """Prostsze przetwarzanie - usunięcie None"""
        print(f"  [{self.nazwa}] Przetwarzam dane...")
        return [x for x in dane if x is not None]

class Ewaluator:
    def __init__(self):
        self.nazwa = "Ewaluator"

    def ocen(self, prawdziwe, przewidywane):
        """Oblicza błąd średniokwadratowy (MSE)"""
        print(f"  [{self.nazwa}] Obliczam MSE...")
        if len(prawdziwe) != len(przewidywane):
            return None
        mse = sum((p - t)**2 for p, t in zip(prawdziwe, przewidywane)) /
len(prawdziwe)
        return mse

# Klasa Model używa kompozycji - zawiera Preprocessor i Ewaluator
class ModelML:
    def __init__(self, nazwa):
        """
        Model ML składa się z preprocessora i ewaluatora.
        """
        self.nazwa = nazwa
        self.preprocessor = Preprocessor()  # Kompozycja: ma preprocessor
        self.ewaluator = Ewaluator()        # Kompozycja: ma ewaluator
        self.parametry = None

    def trenuj(self, dane):
        """Symulacja treningu"""
        dane_czysty = self.preprocessor.przetworz(dane)
        print(f"  [Model {self.nazwa}] Trenuję na {len(dane_czysty)}
obserwacjach...")
        # Prosta symulacja: parametr = średnia
        self.parametry = sum(dane_czysty) / len(dane_czysty)
        print(f"  Parametr modelu: {self.parametry:.2f}")

    def przewiduj(self, x):
        """Prosta predykcja"""
        if self.parametry is None:
            print("Model nie jest wytrenowany!")
            return None
```

```
        return self.parametry * x

    def ocen_model(self, prawdziwe, przewidywane):
        """Wykorzystuje wbudowany ewaluator"""
        return self.ewaluator.ocen(prawdziwe, przewidywane)

# Użycie
model = ModelML("Regresja Prosta")
model.trenuj([10, 20, None, 30, 40, None, 50])

predykcje = [model.przewiduj(1), model.przewiduj(2), model.przewiduj(3)]
prawdziwe = [30, 60, 90]
mse = model.ocen_model(prawdziwe, predykcje)
print(f"\nMSE: {mse:.2f}")
```

Wyjaśnienie:

- `ModelML` **ma** `Preprocessor` i `Ewaluator` jako atrybuty
- Każdy komponent może być używany przez model bez dziedziczenia
- Łatwo można podmienić `Preprocessor` na inny, bardziej zaawansowany

Przykład 3: Kompozycja vs Dziedziczenie – wybór podejścia

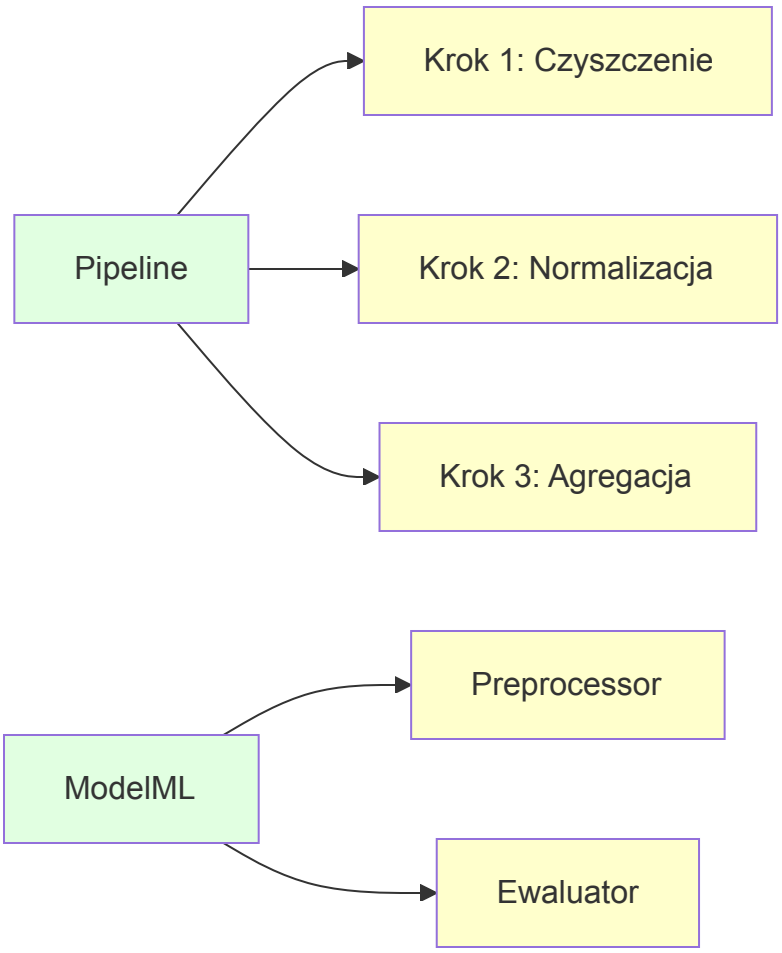
```
# PODEJŚCIE 1: Dziedziczenie (może prowadzić do problemów)
class ModelZPreprocessingiem:
    def preprocessuj(self, dane):
        return [x for x in dane if x > 0]

    def trenuj(self, dane):
        dane_czysty = self.preprocessuj(dane)
        print(f"Trenuję na {len(dane_czysty)} obserwacjach")

# PODEJŚCIE 2: Kompozycja (elastyczne, łatwiejsze w utrzymaniu)
class Preprocessor2:
    def preprocessuj(self, dane):
        return [x for x in dane if x > 0]

class ModelZKomponentami:
    def __init__(self, preprocessor):
        """
        Model przyjmuje preprocessor jako parametr — możemy podać dowolny!
        """
        self.preprocessor = preprocessor

    def trenuj(self, dane):
```

```
        dane_czysty = self.preprocessor.preprocessuj(dane)
        print(f"Trenuję na {len(dane_czysty)} obserwacjach")

# Użycie
preproc = Preprocessor2()
model = ModelZKomponentami(preproc)
model.trenuj([10, -5, 20, -3, 30])
```

Wyjaśnienie:

- **Dziedziczenie**: Model "jest" preprocessorem – sztywne połączenie
- **Kompozycja**: Model "ma" preprocessor – możemy łatwo podmienić na inny
- Kompozycja daje większą elastyczność i łatwiej testować kod

Schemat kompozycji



Tip

Zasada: Prefer composition over inheritance

- Używaj kompozycji, gdy chcesz elastyczności i możliwości podmiany komponentów

- Używaj dziedziczenia, gdy rzeczywiście masz hierarchię typów (np. `ModelRegresji` jest specjalnym typem `Model`)

✓ **Success**

Best practices w Data Science:

- Pipeline'y ML buduj z komponentów (kompozycja)
- Transformery, preprocessory, evaluatory – osobne klasy
- Łatwo testować każdy komponent osobno
- Łatwo wymieniać komponenty bez zmiany całego systemu

🌱 Aspekt środowiskowy

W tej lekcji zwracamy uwagę na:

- **Modularność kodu redukuje zużycie energii**: Kod obiektowy jest łatwiejszy w utrzymaniu, testowaniu i debugowaniu. Mniej błędów = mniej ponownych uruchomień = mniej zużytej energii.
- **Reużywalność komponentów**: Komponenty (klasy, metody) można używać wielokrotnie bez powielania kodu. Powielanie kodu prowadzi do większych plików, dłuższego czasu kompilacji i większego zużycia pamięci.
- **Optymalizacja poprzez kompozycję**: Możesz łatwo podmieniać komponenty pipeline'a na bardziej efektywne bez przepisywania całego systemu. Np. zamiast przetwarzać wszystkie dane, możesz użyć komponentu do próbkowania.

Przykład: Efektywne przetwarzanie danych

```
# Komponent do próbkowania danych (zmniejsza zużycie zasobów)
class Probkowanie:
    def __init__(self, procent=10):
        """
        Komponent do próbkowania danych.
        Zmniejsza rozmiar datasetu, oszczędzając pamięć i czas obliczeń.
        """
        self.procent = procent
        self.nazwa = f"Próbkowanie {procent}%"

    def wykonaj(self, dane):
```

```
        """Zwraca tylko procent% danych"""
        n = max(1, len(dane) * self.procent // 100)
        print(f"  [{self.nazwa}] Redukuję dane z {len(dane)} do {n}
obserwacji")
        # Prosta próbka co k-ty element
        krok = max(1, len(dane) // n)
        return dane[::krok]

# Pipeline z próbkowaniem dla oszczędności zasobów
pipeline_eco = Pipeline("Eco-Pipeline")
pipeline_eco.dodaj_krok(Probkowanie(procent=20))  # Tylko 20% danych
pipeline_eco.dodaj_krok(Normalizacja())

dane_duze = list(range(1, 101))  # 100 obserwacji
wynik_eco = pipeline_eco.uruchom(dane_duze)

# Porównanie: pipeline bez próbkowania przetwarzałby 100 obs,
# z próbkowaniem tylko 20 obs = 80% oszczędności zasobów
```

Wyjaśnienie:

- Dodajemy komponent `Probkowanie` na początku pipeline'a
- Redukujemy rozmiar danych o 80% → mniej pamięci, mniej obliczeń
- W fazie testowania kodu zawsze używaj próbkowania!

✓ **Success**

Green IT w OOP:

- Twórz komponenty wielokrotnego użytku (nie powielaj kodu)
- Buduj pipeline'y modularne – łatwo dodać krok próbkowania
- Testuj kod na małych próbkach przed pełnym uruchomieniem
- Monitoruj zużycie pamięci przez komponenty (użyj biblioteki `psutil`)

🤖 Narzędzia AI/ML wykorzystujące OOP

W ekosystemie Data Science większość bibliotek opiera się na programowaniu obiektowym:

Scikit-learn – estimatory i transformery

```
# Scikit-learn używa OOP:
# - Każdy model to klasa (np. LinearRegression)
# - Każdy transformer to klasa (np. StandardScaler)
# - Pipeline to klasa łącząca komponenty

# Przykład (pseudo-kod - scikit-learn nie jest jeszcze poznany):
# from sklearn.preprocessing import StandardScaler
# from sklearn.linear_model import LinearRegression
# from sklearn.pipeline import Pipeline
#
# pipeline = Pipeline([
#     ('scaler', StandardScaler()),      # Komponent 1
#     ('regressor', LinearRegression())  # Komponent 2
# ])
#
# pipeline.fit(X_train, y_train)
# predictions = pipeline.predict(X_test)
```

Pandas – DataFrame jako klasa

```
# W bibliotece Pandas, DataFrame to klasa:
# - Atrybuty: columns, index, shape, dtypes
# - Metody: head(), describe(), groupby(), merge()

# Przykład (pseudo-kod - Pandas nie jest jeszcze poznany):
# import pandas as pd
#
# df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})  # Obiekt klasy
DataFrame
# print(df.shape)      # Atrybut
# df.describe()        # Metoda
```

Dlaczego to ważne?

- Rozumienie OOP pozwala Ci korzystać z tych bibliotek efektywnie
- Możesz tworzyć własne transformery i estimatory zgodne z API scikit-learn
- Możesz rozszerzać istniejące klasy (np. własny preprocessor dziedziczący po `BaseEstimator`)

📊 Praca z Prawdziwymi Danymi

Dataset: Symulacja datasetu klientów

W tym przykładzie stworzymy prosty system OOP do zarządzania danymi klientów w kontekście e-commerce.

```
# Klasa reprezentująca pojedynczego klienta
class Klient:
    def __init__(self, id, imie, wiek, wydatki):
        """
        Pojedynczy klient w systemie.

        Parameters:
        - id: unikalny identyfikator
        - imie: imię klienta
        - wiek: wiek klienta
        - wydatki: lista wydatków (transakcji)
        """
        self.id = id
        self.imie = imie
        self.wiek = wiek
        self.wydatki = wydatki  # Lista kwot

    def suma_wydatkow(self):
        """Zwraca całkowitą sumę wydatków"""
        return sum(self.wydatki)

    def srednie_wydatki(self):
        """Zwraca średnie wydatki na transakcję"""
        if len(self.wydatki) == 0:
            return 0
        return self.suma_wydatkow() / len(self.wydatki)

    def __str__(self):
        """Reprezentacja tekstowa obiektu (wywoływana przez print())"""
        return f"Klient {self.id}: {self.imie}, {self.wiek} lat, wydał
{self.suma_wydatkow()} zł"

# Klasa zarządzająca bazą klientów (kompozycja - ma listę obiektów Klient)
class BazaKlientow:
    def __init__(self):
        """Baza klientów jako lista obiektów Klient"""
        self.klienci = []

    def dodaj_klienta(self, klient):
        """Dodaje klienta do bazy"""
        self.klienci.append(klient)
```

```python
    def znajdz_klienta(self, id):
        """Znajduje klienta po ID"""
        for klient in self.klienci:
            if klient.id == id:
                return klient
        return None

    def sredni_wiek(self):
        """Oblicza średni wiek wszystkich klientów"""
        if len(self.klienci) == 0:
            return 0
        return sum(k.wiek for k in self.klienci) / len(self.klienci)

    def top_klienci(self, n=5):
        """Zwraca n klientów z największymi wydatkami"""
        posortowani = sorted(self.klienci, key=lambda k: k.suma_wydatkow(),
reverse=True)
        return posortowani[:n]

    def raport(self):
        """Generuje raport o bazie klientów"""
        print(f"=== Raport Bazy Klientów ===")
        print(f"Liczba klientów: {len(self.klienci)}")
        print(f"Średni wiek: {self.sredni_wiek():.1f} lat")
        print(f"Łączne wydatki: {sum(k.suma_wydatkow() for k in
self.klienci)} zł")
        print(f"\nTop 3 klienci:")
        for i, klient in enumerate(self.top_klienci(3), 1):
            print(f"  {i}. {klient}")

# Tworzenie bazy klientów
baza = BazaKlientow()

# Dodawanie klientów
baza.dodaj_klienta(Klient(1, "Anna", 28, [120, 350, 80, 200]))
baza.dodaj_klienta(Klient(2, "Jan", 35, [500, 600, 450]))
baza.dodaj_klienta(Klient(3, "Katarzyna", 42, [100, 150, 200, 180, 220]))
baza.dodaj_klienta(Klient(4, "Piotr", 29, [80, 90, 70]))
baza.dodaj_klienta(Klient(5, "Maria", 51, [300, 400, 500, 600, 700]))

# Generowanie raportu
baza.raport()

# Wyszukiwanie klienta
klient = baza.znajdz_klienta(3)
```

```
if klient:
    print(f"\nZnaleziono: {klient}")
    print(f"Średnie wydatki na transakcję: {klient.srednie_wydatki():.2f}
zł")
```

Wyjaśnienie:

- `Klient` to klasa reprezentująca pojedynczego klienta
- `BazaKlientow` to klasa zarządzająca kolekcją klientów (kompozycja)
- Każdy klient to osobny obiekt z własnymi danymi i metodami
- Baza oferuje metody analityczne: `sredni_wiek()`, `top_klienci()`, `raport()`

Analiza i Preprocessing z OOP

```
# Klasa do analizy danych klientów
class AnalizatorKlientow:
    def __init__(self, baza):
        """
        Analizator operuje na obiekcie BazaKlientow.
        """
        self.baza = baza

    def segmentuj_po_wieku(self):
        """Segmentacja klientów na grupy wiekowe"""
        mlodzi = []      # < 30
        sredni = []      # 30-50
        starsi = []      # > 50

        for klient in self.baza.klienci:
            if klient.wiek < 30:
                mlodzi.append(klient)
            elif klient.wiek <= 50:
                sredni.append(klient)
            else:
                starsi.append(klient)

        print("=== Segmentacja po wieku ===")
        print(f"Młodzi (<30): {len(mlodzi)} klientów")
        print(f"Średni (30-50): {len(sredni)} klientów")
        print(f"Starsi (>50): {len(starsi)} klientów")

        return {"mlodzi": mlodzi, "sredni": sredni, "starsi": starsi}

    def wykryj_vip(self, prog=1000):
        """Wykrywa klientów VIP (wydatki > prog)"""
```

```
        vip = [k for k in self.baza.klienci if k.suma_wydatkow() > prog]
        print(f"\n=== Klienci VIP (wydatki > {prog} zł) ===")
        for klient in vip:
            print(f"  {klient.imie}: {klient.suma_wydatkow()} zł")
        return vip

# Użycie analizatora
analizator = AnalizatorKlientow(baza)
segmenty = analizator.segmentuj_po_wieku()
vip = analizator.wykryj_vip(prog=1000)
```

Wyjaśnienie:

- `AnalizatorKlientow` przyjmuje obiekt `BazaKlientow` jako parametr (kompozycja)
- Oferuje metody analityczne: segmentację, wykrywanie VIP
- Separacja odpowiedzialności: `BazaKlientow` zarządza danymi, `AnalizatorKlientow` analizuje

Wizualizacja Eksploracyjna

```
# Pseudo-wizualizacja (bez matplotlib, używamy printu)
class WizualizatorKlientow:
    def __init__(self, baza):
        self.baza = baza

    def histogram_wydatkow(self):
        """Prosty histogram sumarycznych wydatków (tekstowy)"""
        print("\n=== Histogram Wydatków ===")
        for klient in sorted(self.baza.klienci, key=lambda k:
k.suma_wydatkow()):
            bar = '*' * (klient.suma_wydatkow() // 100)  # 1 gwiazdka = 100
zł
            print(f"{klient.imie:12s} [{klient.suma_wydatkow():4d} zł]
{bar}")

# Użycie wizualizatora
wizualizator = WizualizatorKlientow(baza)
wizualizator.histogram_wydatkow()
```

Wyjaśnienie:

- `WizualizatorKlientow` to kolejny komponent operujący na `BazaKlientow`
- Każda klasa ma jedną odpowiedzialność (Single Responsibility Principle)
- Łatwo testować i rozszerzać każdy komponent osobno

🧪 Zadania do samodzielnej pracy

✏️ Zadania podstawowe (1-8)

1. ✏️ **Zadanie 1 – Tworzenie klasy Produkt**

   Stwórz klasę `Produkt` z atrybutami: `nazwa`, `cena`, `ilosc_w_magazynie`.

   Dodaj metody:

   - `pokaz_info()` – wyświetla informacje o produkcie
   - `sprzedaj(ilosc)` – zmniejsza `ilosc_w_magazynie` o podaną wartość
   - `dostawa(ilosc)` – zwiększa `ilosc_w_magazynie`

   Stwórz 3 obiekty i przetestuj metody.

   (proste)

2. ✏️ **Zadanie 2 – Klasa Kalkulator**

   Stwórz klasę `Kalkulator` z metodami: `dodaj(a, b)`, `odejmij(a, b)`, `pomnoz(a, b)`, `podziel(a, b)`.

   Stwórz obiekt i wykonaj przykładowe obliczenia.

   (proste)

3. ✏️ **Zadanie 3 – Klasa BankAccount**

   Stwórz klasę `BankAccount` z atrybutami: `wlasciciel`, `saldo`.

   Metody:

   - `wplac(kwota)` – zwiększa saldo
   - `wyplac(kwota)` – zmniejsza saldo (tylko jeśli wystarczy środków)
   - `pokaz_saldo()` – wyświetla saldo

   Stwórz kilka kont i przetestuj operacje.

   (proste)

4. ✏️ **Zadanie 4 – Klasa Ksiazka**

   Stwórz klasę `Ksiazka` z atrybutami: `tytul`, `autor`, `rok_wydania`, `liczba_stron`.

   Dodaj metodę `__str__()`, która zwraca czytelną reprezentację książki.

   Stwórz listę 5 książek i wyświetl je używając `print()`.

(proste)

5. ✏️ **Zadanie 5 – Klasa Student**

   Stwórz klasę `Student` z atrybutami: `imie`, `numer_indeksu`, `oceny` (lista ocen).

   Metody:

   - `dodaj_ocene(ocena)` – dodaje ocenę do listy
   - `srednia()` – zwraca średnią ocen
   - `__str__()` – reprezentacja tekstowa

   Stwórz 3 studentów, dodaj im oceny i wyświetl średnie.

   (proste)

6. ✏️ **Zadanie 6 – Dziedziczenie – Pojazd i Samochod**

   Stwórz klasę bazową `Pojazd` z atrybutami: `marka`, `model`, `rok`.

   Stwórz klasę potomną `Samochod` z dodatkowym atrybutem `liczba_drzwi`.

   Dodaj metodę `pokaz_info()` w obu klasach.

   Stwórz obiekt `Samochod` i wywołaj metodę.

   (proste)

7. ✏️ **Zadanie 7 – Klasa Pracownik**

   Stwórz klasę `Pracownik` z atrybutami: `imie`, `stanowisko`, `pensja`.

   Metody:

   - `podwyzka(procent)` – zwiększa pensję o procent%
   - `__str__()` – reprezentacja tekstowa

   Stwórz 3 pracowników i daj im podwyżki.

   (proste)

8. ✏️ **Zadanie 8 – Klasa Zadanie (TODO list)**

   Stwórz klasę `Zadanie` z atrybutami: `opis`, `status` (domyślnie "do zrobienia").

   Metody:

   - `oznacz_jako_zrobione()` – zmienia status na "zrobione"
   - `__str__()` – wyświetla opis i status

Stwórz listę 5 zadań i oznacz część jako zrobione.

(proste)

✏️ Zadania średnie (9-12)

9. ✏️ **Zadanie 9 – Pipeline z wieloma krokami**

   Stwórz 4 komponenty:

   - `FiltrowanieDodatnich` – usuwa liczby ujemne
   - `Zaokraglanie` – zaokrągla do całkowitych
   - `Skalowanie` – mnoży przez stały współczynnik
   - `Sumowanie` – zwraca sumę listy

   Stwórz klasę `Pipeline` i przetestuj na danych: `[1.7, -3.2, 5.8, -1.1, 8.3, 2.4]`

   (średnie)

10. ✏️ **Zadanie 10 – Klasa Zamowienie z kompozycją**

    Stwórz klasę `Produkt` (nazwa, cena) oraz klasę `Zamowienie`, która zawiera listę produktów.

    Metody `Zamowienie`:

    - `dodaj_produkt(produkt, ilosc)`
    - `laczna_wartosc()` – suma wartości zamówienia
    - `pokaz_zamowienie()` – wyświetla wszystkie produkty i łączną wartość

    Stwórz zamówienie z 5 produktami.

    (średnie)

11. ✏️ **Zadanie 11 – System ocen uczniów**

    Stwórz klasy:

    - `Uczen` (imie, oceny=[])
    - `Klasa` (nazwa, uczniowie=[])

    Metody `Klasa`:

    - `dodaj_ucznia(uczen)`
    - `srednia_klasy()` – średnia ocen wszystkich uczniów
    - `najlepszy_uczen()` – zwraca ucznia z najwyższą średnią

Stwórz klasę z 5 uczniami i oblicz statystyki.

(średnie)

12. ✏️ **Zadanie 12 – Dziedziczenie – FiguraGeometryczna**

    Stwórz klasę bazową `Figura` z metodą `pole()`.

    Stwórz klasy potomne:

    - `Prostokat(szerokosc, wysokosc)`
    - `Kolo(promien)`

    Każda klasa implementuje `pole()`.

    Stwórz listę figur i oblicz łączne pole.

    (średnie)

🧠 Zadania wyzwanie (13-20)

13. 🧠 **Zadanie 13 – System biblioteki**

    Stwórz system zarządzania biblioteką:

    - Klasa `Ksiazka` (tytul, autor, dostepna=True)
    - Klasa `Uzytkownik` (imie, wypozyczone_ksiazki=[])
    - Klasa `Biblioteka` (ksiazki=[], uzytkownicy=[])

    Metody `Biblioteka`:

    - `dodaj_ksiazke(ksiazka)`
    - `zarejestruj_uzytkownika(uzytkownik)`
    - `wypozycz(uzytkownik, ksiazka)` – zmienia `dostepna=False`, dodaje do `wypozyczone_ksiazki`
    - `zwroc(uzytkownik, ksiazka)` – odwrotna operacja
    - `raport()` – wyświetla statystyki

    Zasymuluj wypożyczenia i zwroty dla 3 użytkowników i 10 książek.

    (challenge)

14. 🧠 **Zadanie 14 – Pipeline ML z różnymi transformerami**

    Zaimplementuj:

    - `LogTransformer` – log(x + 1)

- `OutlierRemover` – usuwa wartości > 3*std od średniej
- `BinningTransformer` – dzieli na przedziały (bins)

Stwórz pipeline i przetestuj na danych: `[1, 5, 10, 100, 2, 3, 8, 200, 4, 6]`

(challenge)

15. 🧠 **Zadanie 15 – Symulator portfela inwestycyjnego**

    Klasy:

    - `Akcja` (nazwa, cena_kupna, cena_aktualna)
    - `Portfel` (akcje=[], kapital_poczatkowy)

    Metody `Portfel`:

    - `kup_akcje(akcja, ilosc)`
    - `sprzedaj_akcje(akcja, ilosc)`
    - `aktualizuj_ceny(slownik_cen)` – aktualizuje `cena_aktualna`
    - `wartosc_portfela()` – łączna wartość
    - `zysk_strata()` – różnica względem kapitału początkowego

    Zasymuluj kupno/sprzedaż 5 akcji z różnymi zmianami cen.

    (challenge)

16. 🧠 **Zadanie 16 – Dziedziczenie wielopoziomowe – Zwierzęta**

    Struktura:

    - `Zwierze` (nazwa, wiek)
    - `Ssak(Zwierze)` (typ_futra)
    - `Pies(Ssak)` (rasa)

    Każda klasa ma metodę `pokaz_info()` wyświetlającą wszystkie atrybuty.

    Stwórz obiekt `Pies` i wywołaj metodę.

    (challenge)

17. 🧠 **Zadanie 17 – System rezerwacji hotelu**

    Klasy:

    - `Pokoj` (numer, typ, cena_za_noc, zajety=False)
    - `Rezerwacja` (pokoj, data_od, data_do, gosc)
    - `Hotel` (pokoje=[], rezerwacje=[])

Metody `Hotel`:

- `dodaj_pokoj(pokoj)`
- `rezerwuj(numer_pokoju, gosc, data_od, data_do)` – tworzy rezerwację
- `wolne_pokoje()` – lista wolnych pokoi
- `przychod()` – suma przychodów ze wszystkich rezerwacji

Zasymuluj 10 pokoi i 5 rezerwacji.

(challenge)

18. 🧠 **Zadanie 18 – Pipeline z walidacją danych**

Stwórz komponenty:

- `ValidatorZakresu(min, max)` – sprawdza, czy wartości w zakresie
- `ValidatorTypu(typ)` – sprawdza, czy wszystkie wartości są danego typu
- `ValidatorBrakow` – sprawdza, czy nie ma None/NaN

Każdy validator zwraca `(True/False, komunikat)`.

Pipeline agreguje wyniki walidacji.

(challenge)

19. 🧠 **Zadanie 19 – Gra RPG z klasami postaci**

Klasy:

- `Postac` (imie, hp, atak, obrona)
- `Wojownik(Postac)` – zwiększony atak
- `Mag(Postac)` – umiejętność rzucania czarów
- `Lucznik(Postac)` – zwiększona szybkość

Metody:

- `atakuj(cel)` – zadaje obrażenia
- `leczenie(punkty)` – przywraca HP

Zasymuluj walkę 3 postaci.

(challenge)

20. 🧠 **Zadanie 20 – Ekosystem transformerów danych**

Stwórz hierarchię:

- `BaseTransformer` (klasa bazowa z metodą `transform(dane)`)

- `DropNullTransformer` – usuwa None
- `ClipTransformer(min, max)` – ogranicza wartości do zakresu
- `SquareTransformer` – podnosi do kwadratu
- `InverseTransformer` – 1/x

Stwórz `Pipeline` z 4 transformerami i przetestuj na danych.

Dodaj metodę `pipeline.fit_transform(dane)` analogiczną do scikit-learn.

(challenge)

📚 Podsumowanie

W tej lekcji nauczyłeś się:

- **Podstaw OOP**: klasy, obiekty, atrybuty, metody, konstruktor `__init__`, referencja `self`
- **Dziedziczenia**: tworzenie hierarchii klas, `super()`, nadpisywanie metod
- **Kompozycji**: budowanie klas z komponentów, relacja "ma" vs "jest"
- **Best practices**: kiedy używać dziedziczenia, a kiedy kompozycji
- **Zastosowań w Data Science**: pipeline'y, transformery, modele, komponenty analityczne
- **Aspektów środowiskowych**: modularność kodu = mniej energii, reużywalność komponentów, optymalizacja przez próbkowanie

Połączenie z poprzednimi lekcjami:

- Wykorzystujesz listy, słowniki, pętle z poprzednich lekcji
- Używasz funkcji do budowania metod w klasach
- Stosujesz logikę warunkową w metodach

Przygotowanie do następnych tematów:

- NumPy i Pandas opierają się na OOP (DataFrame to klasa!)
- Scikit-learn używa klas do modeli i transformerów
- Pipeline'y ML buduje się z komponentów (kompozycja)

Refleksja nad aspektem środowiskowym:

- Kod obiektowy jest łatwiejszy w utrzymaniu → mniej błędów → mniej ponownych uruchomień → oszczędność energii
- Komponenty wielokrotnego użytku → mniej powielania kodu → mniejsze pliki → mniej pamięci

- Pipeline'y modularne → łatwo dodać krok próbkowania danych → oszczędność zasobów

---

📖 Dodatkowe Zasoby

- Dokumentacja Python – Classes – oficjalny tutorial OOP w Pythonie
- Real Python – OOP in Python – szczegółowy przewodnik po OOP
- Composition vs Inheritance – kiedy używać której techniki
- Design Patterns in Python – wzorce projektowe (zaawansowane)
- Scikit-learn API Design – jak budować transformery zgodne z scikit-learn
- Clean Code in Python – best practices w kodzie OOP

Polecane datasety do praktyki:

- Własne dane biznesowe (klienci, transakcje, produkty)
- Symulowane dane (generator Pythonowy)
- Proste CSV do wczytania w przyszłych lekcjach (po poznaniu Pandas)